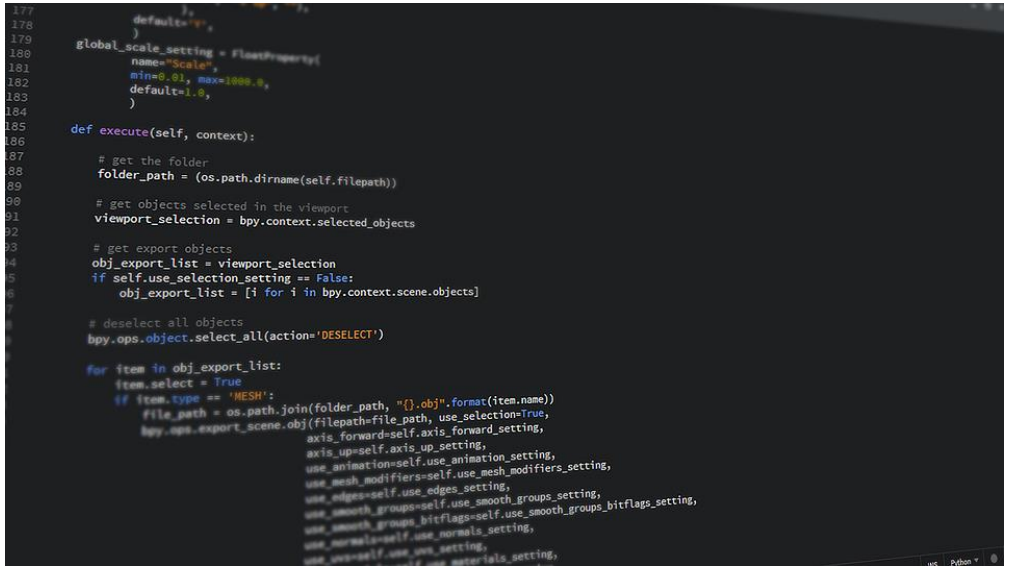
Einführung in Python

-- Funktionen, Kommentare, Funktionen als Variablen, Funktionen höherer Ordnung -

Kristof Van Laerhoven
kvl@eti.uni-siegen.de

Michael Möller
michael.moeller@uni-siegen.de

Andreas Hoffmann
andreas.hoffmann@uni-siegen.de



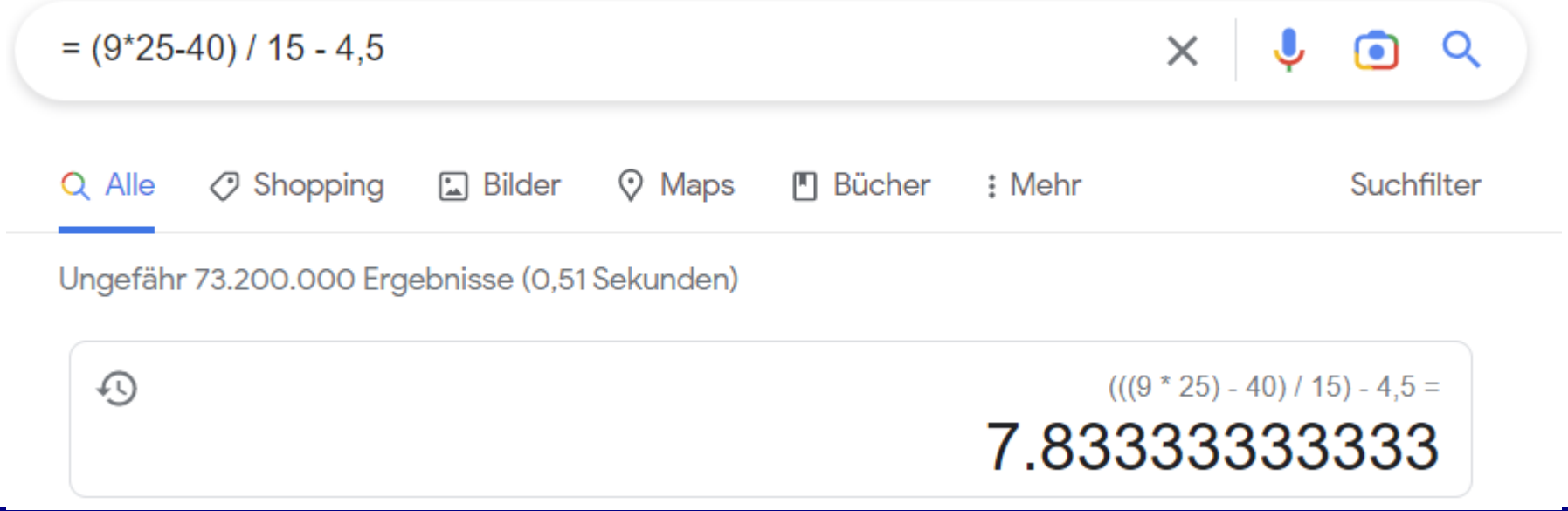
Feedback

https://crs.zimt.uni-siegen.de/

Erste Hinweise

Wenn etwas fehlt (z.B. auf Moodle) dann müssen sie sich melden!

Sie werden Vorlesungen mehrere Stunden nachbereiten müssen.



Jetzt mit mehr Zeit: for-Schleifen


```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

Im Allgemeinen hat `range` einen Startpunkt, einen Endpunkt, und eine Schrittweite

```
for i in range(1,10,3):
    print(i)
```

```
1
4
7
```

Python können `for`-Schleifen aber auch noch allgemeiner, nämlich für beliebige *iterierbare Objekte* verwendet werden. Unter den uns bekannten Objekten, sind dies z.B. Listen:

```
meine_liste = ['hallo', 4, [1,2,3]]
for element in meine_liste:
    print(element)
```

```
hallo
4
[1, 2, 3]
```

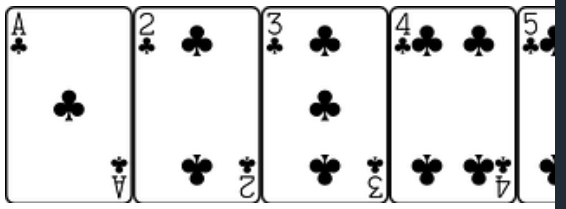
Mir `enumerate` kann zusätzlich auf den Index zugegriffen werden:

```
meine_liste = ['hallo', 4, [1,2,3]]
for index, element in enumerate(meine_liste):
    print('An index {} steht {}'.format(index, element))
```

```
An index 0 steht hallo
An index 1 steht 4
An index 2 steht [1, 2, 3]
```

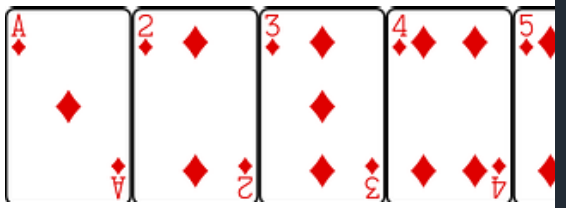
UNIVERSITÄT SIEGEN

Spieler 1



Preiskarten

Spieler 2



Beim letzten Mal: Geschafft dieses Programm sinnvoll zum Laufen zu bringen. Aber ist dies *gut* programmiert?

```python
from random import shuffle
kartenSpieler1 =   [1,2,3,4,5,6,7,8,9,10,11,12,13] # das hier werden Listen an Integers;
langfristig ersetzen durch list(1,range(n+1)), mit n=13
#kartenSpieler2 =  [1,2,3,4,5,6,7,8,9,10,11,12,13]  # no need: KI-2 always play the price
card as its choice
preiskarten =  [1,2,3,4,5,6,7,8,9,10,11,12,13]
shuffle(preiskarten)

### "KI"-1: Spiele eine zufällige Karte:
shuffle(kartenSpieler1)

punkteSpieler1 = 0
punkteSpieler2 = 0

while (len(preiskarten)>0):  # man könnte auch nur while (preiskarten): schreiben.

    neuePreiskarte = preiskarten.pop()#Neue Preiskarte zufällig ziehen
    neueKarteSpieler1 =kartenSpieler1.pop()
    neueKarteSpieler2 = neuePreiskarte ### Vorsicht: hier müsste man aufpassen wenn sich
andere Teile des Programms ändern, z.B. andere Preiskarten als Spielkarten.

    if (neueKarteSpieler1 > neueKarteSpieler2):
        punkteSpieler1 = punkteSpieler1 + neuePreiskarte
        print("Spieler 1 gewinnt die Runde. Neuer Punktestand: ", punkteSpieler1, "vs.",
punkteSpieler2)
    elif (neueKarteSpieler2 > neueKarteSpieler1):
        punkteSpieler2 = punkteSpieler2 + neuePreiskarte
        print("Spieler 2 gewinnt die Runde. Neuer Punktestand: ", punkteSpieler1, "vs.",
punkteSpieler2)
    else:
        print("Gleichstand - nichts passiert! Neuer Punktestand: ", punkteSpieler1, "vs.",
punkteSpieler2)

28

if (punkteSpieler1 > punkteSpieler2):
    print('Spieler 1 hat gewonnen!')
elif (punkteSpieler2 > punkteSpieler1):
    print('Spieler 2 hat gewonnen!')
else:
    print('Spieler 1 und Spieler 2 haben gleich viele Punkte!')
```

Nächster Schritt

Existierenden Code etwas besser machen.

Funktionen

Mit größer werdenden Programmen wird es extrem schwierig Code zu verstehen und zu pflegen, wenn alles in einer Datei sequenziell hintereinander geschrieben ist. Ein deutlich besserer Stil ist das aufteilen in Sinnabschnitte mittels *Funktionen*.

```
def name_der_function(beliebig, viele, argumente):
        """
        Dies ist ein docstring. Hier kann dokumentiert werden was die Funktion tut
        """

        aller eingerückter Code wird ausgeführt

        return rueckgabewert  (falls gewünscht)
```

```
def meine_funktion():
    print('hallo')

meine_funktion()
```

```
In [1]: runfile('temp.py')
hallo
```

```
def meine_funktion(name):
    print('hallo', name)

meine_funktion('zusammen!')
```

```
In [3]: runfile('temp.py')
hallo zusammen!
```

Funktionen - Beispiel

Wir können in unserem Goofspiel einige Teile als Funktionen auslagern!
Beispiele hierfür im Code!

Funktionen


```
def strategie_spieler_1(wert_der_preiskarte):
    wert_karte_1 = karten_spieler_1.pop()
    print('Spieler 1 spielt eine ', wert_karte_1)
    return wert_karte_1

def strategie_spieler_2(wert_der_preiskarte):
    wert_karte_2 = wert_der_preiskarte
    karten_spieler_2.remove(wert_karte_2)
    print('Spieler 2 spielt eine ', wert_karte_2)
    return wert_karte_2

while (preiskarten):
    wert_der_preiskarte = preiskarten.pop()
    print('\nDer Wert der Preiskarte ist ',wert_der_preiskarte)

    wert_karte_1 = strategie_spieler_1(wert_der_preiskarte)

    wert_karte_2 = strategie_spieler_2(wert_der_preiskarte)
```

Ein solcher Code ist schon eine signifikante Verbesserung: Wir haben die Strategien der Spieler in eigenen Funktionen isoliert. Ein Programmierer kann nun die Strategie ändern ohne das Hauptprogramm vollständig durchzuarbeiten oder zu verstehen! Um solche modularen Änderungen einfacher möglich zu machen, sollte die Funktionsweise von Funktionen mit dokumentiert werden. Dies geschieht über den *docstring*.

Funktionen mit Docstring


```python
def auswertung_am_spielende(punkte_spieler_1,punkte_spieler_2):
    """
    Parameters
    ----------
    punkte_spieler_1 : Anzahl der Punkte, die Spieler 1
                       insgesammt erreicht hat.
    punkte_spieler_2 : Anzahl der Punkte, die Spieler 1
                       insgesammt erreicht hat.

        Vergleicht die Anzahl der Punkte und gibt eine Nachricht über den
        Gewinner oder ein Unentschieden auf der Konsole aus.

    Returns
    -------
    None.
    """

    if punkte_spieler_1 > punkte_spieler_2:
        print("Spieler 1 hat gewonnen!")
    elif punkte_spieler_1 < punkte_spieler_2:
        print("Spieler 2 hat gewonnen!")
    else:
        print('Dieses Spiel war unentschieden!')
```

Docstring der Funktion


```
In [20]: help(auswertung_am_spielende)
Help on function auswertung_am_spielende in module __main__:

auswertung_am_spielende(punkte_spieler_1, punkte_spieler_2)
    Parameters
    ----------
    punkte_spieler_1 : Anzahl der Punkte, die Spieler 1
                       insgesammt erreicht hat.
    punkte_spieler_2 : Anzahl der Punkte, die Spieler 1
                       insgesammt erreicht hat.

        Vergleicht die Anzahl der Punkte und gibt eine Nachricht über den
        Gewinner oder ein Unentschieden auf der Konsole aus.

    Returns
    -------
    None.
```

Kommentare

Für längere Funktionen (und jeden anderen Code) macht es Sinn über den docstring hinaus noch *Kommentare* einzufügen. Dies geschieht in Python durch das Symbol #:

```
# alles was hier steht wird vom Interpreter ignoriert
```

Dabei sollte darauf geachtet werden, dass sinnvolle (die Logik des Programms erklärende) Kommentare genutzt werden

```
# der Wert der Variablen b wird auf a gesetzt
b = a
```

Hier wurde der Code zwar kommentiert, aber ohne dass der Kommentar irgendeinen Mehrwert liefert!

```
# der Variablen wert_der_preiskarte wird der Rückgabewert
# der auf preiskarten aufgerufenen Funktion pop zugeordnet
wert_der_preiskarte = preiskarten.pop()
```

schlecht

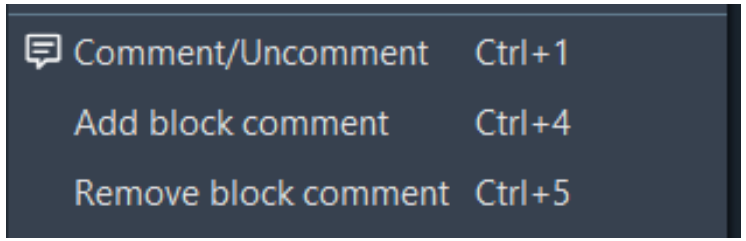
```
# die letzte der Preiskarten wird als wert_der_preiskarte
# verwendet und aus der Liste preiskarten gelöscht.
wert_der_preiskarte = preiskarten.pop()
```

besser

Kommentare

Die meisten Editoren und IDEs haben eine Funktionalität alle gerade markierten Zeilen aus- bzw. einzukommentieren.

Z.B. Spyder unter Windows:



Alternativ kann man die Notation des Docstrings (drei Anführungszeichen) auch für Blockkommentare nutzen. Formal ist dies in Python ein String über mehrere Zeilen, der einfach keiner Variablen zugeordnet wird.

```
"""
Hier wird irgendetwas kommentiert
a = 1
b = 2
c = 3

"""
```

positional arguments vs. keyword arguments

In Python kann man Funktionen (wie vorhin gesehen) mit einer Reihe von Eingabeargumenten versehen.

Beim Aufruf muss dann die Reihenfolge der Eingabeargumente beibehalten werden
– man spricht von *positional arguments*.

```
def meine_funktion(grußwort, name):
    print(grußwort, name)

meine_funktion('Hallo', 'zusammen')
meine_funktion('zusammen', 'Hallo')
```

```
Hallo zusammen
zusammen Hallo
```

Man kann aber den in der Funktion definierten Eingabeargumenten auch default (initial-)Werte geben. In diesem Fall sind sie unter dem in der Definition genannten Namen bekannt, also sogenannte *keyword arguments*, und können mittels ihres Namens in beliebiger Reihenfolge mit Werten versehen werden.

```
def meine_funktion(grußwort='Hallo', name=''):
    print(grußwort, name)

meine_funktion()
meine_funktion(name = 'zusammen')
meine_funktion(grußwort='Hi')
meine_funktion(grußwort='Hi', name='zusammen')
```

```
Hallo
Hallo zusammen
Hi
Hi zusammen
```

Funktionen mit variabler Anzahl an Argumenten

Für beide, positional arguments und keyword arguments, kann man mittels * bzw. ** eine beliebige Anzahl von Eingabeargumenten erlauben!

```
def name_der_function(*args):
        Hier steht eine Funktion, die mit beliebig vielen in args gespeicherten
        Positionalen Argumenten umgehen kann.

def name_der_function(**kwargs):
        Hier steht eine Funktion, die mit beliebig vielen in kwargs gespeicherten keyword
        Argumenten umgehen kann.
```

```
def meine_funktion(*args):
    print(type(args))
    for argument in args:
        print(argument)
meine_funktion('hallo', 4*4, 'zusammen!', [3,2,1])
```

```
def meine_funktion(**kwargs):
    print(type(kwargs))
    for key,value in kwargs.items():
        print('unter dem namen "{}" steht der wert "{}"'.format(key, value))
```

Type Hints

Man kann explizit angeben welche Eingabe und Ausgabetypen eine Funktion bekommt und liefert

```
def name_der_function(variable1: typ1, variable2: typ2) -> ausgabeTyp:
```

Dies geht übrigens auch für Variablen

```
a: int = 3
name: str = “Michael“
meine_liste: list = [1, ‘hallo‘, 5.342]
```

Dies sind Hinweise, die auf den Ablauf des Programms KEINEN Einfluss haben. Insbesondere führt

```
a: int = 3.1415
```

(oder ein späteres Überschreiben eines a: int) nicht zu einem Fehler. Es gibt jedoch Möglichkeiten/Module, die dies können/zum Debuggen verwenden!

Funktionen als Variablen

Eine interessante Eigenschaft von Python ist, dass Funktionen auch als Funktionsobjekte in Variablen gespeichert werden können!

```
def say_hello(name):
    print('Hello', name)

meine_funktion = say_hello
```

```
print(meine_funktion)
```

Hexadezimale Speicheradresse der Funktion

```
<function say_hello at 0x0000016B54407C10>
```

```
meine_funktion('Michael')
```

```
Hello Michael
```

meine_funktion kann nun genauso genutzt werden wie say_hello

```
print(meine_funktion('Michael'))
```

```
Hello Michael
None
```

Da die Funktion say_hello (und damit auch meine_funktion) keinen Rückgabewert hat, wird hier None auf der Konsole ausgegeben.

Funktionen als Variablen - Beispiel

Funktionen als Variablen zu speichern ermöglicht die Konstruktion von Funktionen höherer Ordnung (higher order function), d.h. Funktionen die als ein Eingabeargument eine andere Funktion nehmen.

Beispiel: Schreibe Funktionen zum Addieren und Multiplizieren von Zahlen, sowie eine Funktion „anwenden“ die eine gegebene Funktion auf zwei Zahlen anwendet.

```
def addiere(x,y):
    return x+y

def multipliziere(x,y):
    return x*y

def anwenden(x,y,funktion):
    return meine_funktion(x,y)

meine_funktion = multipliziere
print(anwenden(2,3,meine_funktion))
```

Idee: Die in einer Variablen gespeicherten Funktion könnte sich innerhalb des Programms (z.B. durch Benutzereingaben) ändern.

Funktionen als Variablen - Beispiel

Unser vorheriger Goofspiel-Code hat die Strategien der Spieler (hier letzte Karte spielen bzw. Wert der Preiskarte spielen) immer noch hart festgelegt (siehe unten). Wie könnte man dies (insbesondere bei wachsender Anzahl an KI-Strategien oder menschlichen Inputs) verbessern?

```
def strategie_spieler_1(wert_der_preiskarte):
    wert_karte_1 = karten_spieler_1.pop()
    print('Spieler 1 spielt eine ', wert_karte_1)
    return wert_karte_1

def strategie_spieler_2(wert_der_preiskarte):
    wert_karte_2 = wert_der_preiskarte
    karten_spieler_2.remove(wert_karte_2)
    print('Spieler 2 spielt eine ', wert_karte_2)
    return wert_karte_2

while (preiskarten):
    wert_der_preiskarte = preiskarten.pop()
    print('\nDer Wert der Preiskarte ist ',wert_der_preiskarte)

    wert_karte_1 = strategie_spieler_1(wert_der_preiskarte)

    wert_karte_2 = strategie_spieler_2(wert_der_preiskarte)
```

Funktionen in ausgelagerten Dateien

Mit wachsender Programmgröße ist es oft sinnvoll bestimmte Funktionen in einer Datei (in der nicht das Hauptprogramm steht) zusammenzufassen.

Um diese in einem anderen Programm verwenden zu können, benutzt man den Befehl `import`

Wie schon zuvor bei `import random` gesehen, wird der Name der importierten Datei dann zum *Namespace* für alle darin enthaltenen Funktionen (erinnern sie sich an `random.shuffle(karten)`).

Mit `from (name_des_moduls) import (name_der_funktion)` kann man Funktionen verwenden ohne dass diese in einem Namespace sind.

Mit `import (name_des_moduls) as (name_des_gewünschten_namespaces)` kann man den Namespace der im importierten Modul enthaltenen Funktionen selbst bestimmen (und vom Modulnamen abweichen).

Funktionen in ausgelagerten Dateien - Beispiel

Datei *strategien.py*

```
import random

def spiele_wert_der_preiskarte(karten, wert_der_preiskarte):
    wert_karte = wert_der_preiskarte
    karten.remove(wert_karte)
    return wert_karte

def spiele_zufaellige_karte(karten, wert_der_preiskarte):
    wert_karte = random.sample(karten, 1)[0]
    karten.remove(wert_karte)
    return wert_karte
```

Im Hauptprogramm:

```
import strategien

strategie_spieler_1 = strategien.spiele_zufaellige_karte
strategie_spieler_2 = strategien.spiele_wert_der_preiskarte
```

Funktionen in ausgelagerten Dateien

Beim `import` Befehl muss die Datei im gleichen Ordner wie das ausgeführte Skript sein (oder im PYTHONPATH sein). Für Unterordner kann man (Name des Unterordners) **.** (Name der Datei) verwenden.

Zum Beispiel

```
import subdirectory.strategien as strat

strategie_spieler_1 = strat.spiele_zufaellige_karte
strategie_spieler_2 = strat.spiele_wert_der_preiskarte
```